

LAPORAN PRAKTIKUM STRUKTUR DATA

“ALGORITMA SORTING”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadiran Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-7 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada konsep algoritma Sorting (Insertion Sort, Selection Sort dan Bubble Sort).

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dalam langkah langkah ketiga algoritma Sorting dan dapat mengidentifikasi perbedaan terhadap masing masing algoritma yang dibahas..

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Program InsertionSort_2511531001	2
2.2.1 Kode Program	2
2.2.2 Penjelasan.....	3
2.2.3 Output.....	4
2.3 Program SelectionSort_2511531001.....	4
2.3.1 Kode Program	4
2.3.2 Penjelasan.....	5
2.3.3 Output.....	5
2.4 Program BubbleSort_2511531001	5
2.4.1 Kode Program	5
2.4.2 Penjelasan.....	6
2.4.3 Output.....	7
2.5 Program InsertionGUI_2511531001	7
2.5.1 Kode Program	7
2.5.2 Penjelasan.....	11
2.5.3 Output.....	12
BAB III KESIMPULAN.....	15
DAFTAR PUSTAKA	16
LAPORAN TUGAS.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer, pengurutan data atau sorting merupakan operasi fundamental yang memiliki peran krusial dalam pengorganisasian suatu informasi. Pemilihan pemrograman yang tepat tidak hanya mempengaruhi kecepatan eksekusi program, tetapi juga penggunaan memori dan juga stabilitas data. Pemahaman mendalam terkait mekanisme kerja, karakteristik dan juga kompleksitas waktu dari berbagai pengurutan menjadi kompetensi dasar yang wajib dikuasai dalam pengembangan perangkat lunak.

Praktikum ini dirancang untuk mengimplementasikan dan mengevaluasi kinerja dari beberapa algoritma sorting sehingga kita dapat memverifikasi kebenaran logika algoritma dan mengetahui kompleksitas ruang dan waktu dari masing masing algoritma yang dibahas.

1.2 Tujuan

1. Memahami konsep dasar dari Algoritma Sorting.
2. Mengidentifikasi perbedaan dalam penggunaan jenis jenis Algoritma Sorting.
3. Mengimplementasikan algoritma Sorting pada program yang menggunakan GUI.

1.3 Manfaat

1. Memperdalam pemahaman tentang prinsip dasar pengurutan data dan logika.
2. Melatih kemampuan menerjemahkan logika/pseudocode untuk mengimplementasikan kode yang benar, rapi juga efisien.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting adalah proses mengurutkan data yang semulanya belum terurut menjadi data yang terurut dalam ketentuan tertentu. Algoritma Sorting adalah sekumpulan langkah-langkah yang digunakan untuk mengatur data dalam urutan tertentu. Dalam pemrograman tersendiri, data yang bisa diurutkan bisa berupa angka, huruf (aturan pengurutan diatur sesuai dengan alfabet) dan juga bisa objek kompleks seperti data mahasiswa, produk maupun transaksi. Ada beberapa jenis Algoritma Sorting yang di dalam praktikum ini akan dibahas 3 diantaranya, yaitu ;

1. Insertion Sort : menggunakan prinsip penyisipan kepada posisi yang tepat dan mirip seperti cara mengurutkan kartu remi.
2. Selection Sort : menggunakan prinsip pencarian nilai terkecil dari semua elemen yang ada dan menempatkannya di posisi awal dan diulang untuk elemen berikutnya.
3. Bubble Sort : menggunakan prinsip perbandingan dua elemen bersebelahan dan menukar jika urutannya salah, ,ciri khas dari algoritma ini adalah nilai pada sisi kanan yang terlebih dahulu terurut.

2.2 Program InsertionSort_2511531001

2.2.1 Kode Program

```
package pekan7_2511531001;
public class InsertionSort_2511531001 {
    public static void insertionSort_2511531001 (int[]
arr_1001) {
        int n_1001 = arr_1001.length;
        for (int i_1001 = 1; i_1001 < n_1001;
i_1001++) {
            int key_1001 = arr_1001 [i_1001];
            int j_1001 = i_1001 - 1;
            while (j_1001 >= 0 && arr_1001
[j_1001] > key_1001) {
                arr_1001 [j_1001 + 1] =
arr_1001[j_1001];
                j_1001--;
            }
            arr_1001 [j_1001+1] = key_1001;
```

```

    }
}
public static void main (String [] args) {
    int arr_1001 [] = {23, 78, 45, 8, 32, 56, 1};
    int n_1001 = arr_1001.length;
    System.out.printf("array yang belum
terurut:\n");
    for (int i_1001 = 0; i_1001 < n_1001 ;
i_1001++)
        System.out.print(arr_1001[i_1001] + "
");
        System.out.println("");
        insertionSort_2511531001(arr_1001);
        System.out.printf("array yang terurut:\n");
        for (int i_1001 = 0; i_1001 < n_1001 ;
i_1001++)
            System.out.print(arr_1001 [i_1001] + "
");
            System.out.println("");
        }
}
}

```

Kode Program 2.1 : Kode Program Insertion Sort

2.2.2 Penjelasan

Kode ini dimulai dengan menginisiasi nama method insertionSort dengan array bertipe integer sebagai object methodnya. Lalu panjang array disimpan dalam variabel bertipe n. Setelah itu adanya perulangan yang berfungsi untuk mengurutkan elemen-elemen dalam array arr_1001 secara menaik dari nilai nilai terkecil ke terbesar. Pada perulangan, variabel key_1001 digunakan untuk menyimpan elemen saat ini dan j_1001 adalah variabel yang digunakan sebagai penunjuk ke elemen tepat sebelum elemen yang disimpan dalam variabel key_1001.

Setelah itu program dilanjutkan dengan class main, dalam class ini dimulai dengan memasukkan beberapa nilai ke dalam array_1001, lalu menampilkannya. Setelah itu program memanggil method insertionSort dan menampilkan kembali array yang sudah terurut.

2.2.3 Output

```
<terminated> InsertionSort_2511531001
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.3 Program SelectionSort_2511531001

2.3.1 Kode Program

```
package pekan7_2511531001;
public class SelectionSort_2511531001 {
    public static void selectionSort_2511531001 (int []
arr_1001) {
        int n_1001 = arr_1001.length;
        for (int i_1001 = 0; i_1001 < n_1001 ; i_1001++) {
            int minIndex_1001 = i_1001;
            for (int j_1001 = i_1001 +1; j_1001 <
n_1001 ; j_1001++) {
                if (arr_1001 [j_1001]<
arr_1001[minIndex_1001]) {
                    minIndex_1001 = j_1001;
                }
            }
            int temp_1001 = arr_1001 [i_1001];
            arr_1001 [i_1001] = arr_1001 [minIndex_1001];
            arr_1001 [minIndex_1001] = temp_1001;
        }
    }
    public static void main (String [] args) {
        int arr_1001 [] = {23, 78, 45, 8, 32, 56, 1};
        int n_1001 = arr_1001.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_1001 = 0; i_1001 < n_1001 ; i_1001++)
            System.out.print(arr_1001[i_1001] + " ");
        System.out.println("");
        selectionSort_2511531001(arr_1001);
        System.out.printf("array yang terurut:\n");
        for (int i_1001 = 0; i_1001 < n_1001 ; i_1001++)
            System.out.print(arr_1001 [i_1001] + " ");
        System.out.println("");
    }
}
```



```

arr_1001 [j_1001] = arr_1001
[j_1001+1];
arr_1001 [j_1001+1]=temp_1001;
//System.out.println("data: " +
arr_1001[j_1001] + " "+ arr_1001 [j_1001+1]);
    }
    }
}
}
public static void main (String [] args) {
    int arr_1001 [] = {23, 78, 45, 8, 32, 56, 1};
    int n_1001 = arr_1001.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_1001 = 0; i_1001 < n_1001 ; i_1001++)
        System.out.print(arr_1001[i_1001] + " ");
    System.out.println("");
    //minMaxSelectionSort(arr,n);
    bubbleSort_2511531001(arr_1001);
    System.out.printf("array yang terurut:\n");
    for (int i_1001 = 0; i_1001 < n_1001 ; i_1001++)
        System.out.print(arr_1001 [i_1001] + " ");
    System.out.println("");
}
}
}

```

2.4.2 Penjelasan

Program ini dimulai dengan membuat method bubbleSort dengan arr dengan tipe integer sebagai objeknya. Lalu, pendeklarasian variabel n untuk menyimpan panjang dari array. Lalu untuk perulangan for sebanyak dua kali untuk menggeser bilangan yang paling besar ke bagian kanan dari array.

Lalu kelas main untuk program ini dimulai dengan memasukkan nilai elemen ke dalam array. Lalu menampilkan array sebelum diurutkan dan program akan memanggil method BubbleSort. Setelah itu menampilkan kembali array yang telah diurutkan.

2.4.3 Output

```
<terminated> BubleSort_2511531001 [Ja
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.5 Program InsertionGUI_2511531001

2.5.1 Kode Program

```
package pekan7_2511531001;

import java.awt.BorderLayout;
public class InsertionGUI_2511531001 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_1001;
    private JLabel[] labelArray_1001;
    private JButton stepButton_1001, resetButton_1001,
setButton_1001;
    private JTextField inputField_1001;
    private JPanel panelArray_1001;
    private JTextArea stepArea_1001;

    private int i_1001 = 1, j_1001;
    private boolean sorting_1001 = false;
    private int stepCount_1001 = 1;

    /**
     * Create the frame.
     */
    public InsertionGUI_2511531001() {
        setTitle("Insertion Sort Langkah per
Langkah");
        setSize(750,400);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout (new BorderLayout());

        //panel input
        JPanel inputPanel_1001 = new JPanel (new
FlowLayout());
        inputField_1001 = new JTextField (30);
        setButton_1001 = new JButton ("Set Array");
        inputPanel_1001.add(new JLabel ("Masukkan
angka (pisahkan dengan koma:"));
        inputPanel_1001.add(inputField_1001);
        inputPanel_1001.add(setButton_1001);

        //panel array visual
```

```

        panelArray_1001 = new JPanel();
        panelArray_1001.setLayout (new FlowLayout());

        //panel kontrol
        JPanel controlPanel_1001 = new JPanel ();
        stepButton_1001 = new JButton ("Langkah
Selanjutnya");
        resetButton_1001 = new JButton ("Reset");
        stepButton_1001.setEnabled(false);
        controlPanel_1001.add(stepButton_1001);
        controlPanel_1001.add(resetButton_1001);

        //Area teks untuk log langkah-langkah
        stepArea_1001 = new JTextArea (8,60);
        stepArea_1001.setEditable(false);
        stepArea_1001.setFont(new Font ("Monospaced",
Font.PLAIN,14));
        JScrollPane scrollPane_1001 = new JScrollPane
(stepArea_1001);

        //Tambahkan panel ke frame
        add(inputPanel_1001, BorderLayout.NORTH);
        add(panelArray_1001, BorderLayout.CENTER);
        add(controlPanel_1001, BorderLayout.SOUTH);
        add(scrollPane_1001, BorderLayout.EAST);

        //event Set Array
        setButton_1001.addActionListener ( e ->
setArrayFromInput_1001());

        //event Langkah Selanjutnya
        stepButton_1001.addActionListener ( e->
performStep_1001());

        //event Reset
        resetButton_1001.addActionListener(e ->
reset_1001());
    }

    private void setArrayFromInput_1001() {
        String text_1001 =
inputField_1001.getText().trim();
        if ( text_1001.isEmpty()) return;
        String [] parts_1001 = text_1001.split(",");
        array_1001 = new int [parts_1001.length];

        try {
            for ( int k_1001 = 0; k_1001 <
parts_1001.length; k_1001++) {
                array_1001 [k_1001] =
Integer.parseInt(parts_1001[k_1001].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka yang dipisahkan" + " dengan koma !" ,
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

```

```

    }
    i_1001 = 1;
    stepCount_1001 = 1;
    sorting_1001 = true;
    stepButton_1001.setEnabled(true);
    stepArea_1001.setText("");
    panelArray_1001.removeAll();
    labelArray_1001 = new JLabel
[array_1001.length];
    for (int k_1001 = 0; k_1001 <
array_1001.length; k_1001++) {
        labelArray_1001[k_1001] = new JLabel
(String.valueOf(array_1001[k_1001]));
        labelArray_1001[k_1001].setFont(new Font
("Arial", Font.BOLD, 24));

        labelArray_1001[k_1001].setBorder(BorderFactory.createLineB
order(Color.BLACK));
        labelArray_1001[k_1001].setPreferredSize(new
Dimension (50, 50));

        labelArray_1001[k_1001].setHorizontalAlignment(SwingConstan
ts.CENTER);
        panelArray_1001.add(labelArray_1001[k_1001]);
    }
    panelArray_1001.revalidate();
    panelArray_1001.repaint();
}
/**
 * Create the frame.
 */
public InsertionGUI_2511531001() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout (new BorderLayout());

    //panel input
    JPanel inputPanel_1001 = new JPanel (new
FlowLayout());
    inputField_1001 = new JTextField (30);
    setButton_1001 = new JButton ("Set Array");
    inputPanel_1001.add(new JLabel ("Masukkan angka
(pisahkan dengan koma):"));
    inputPanel_1001.add(inputField_1001);
    inputPanel_1001.add(setButton_1001);

    //panel array visual
    panelArray_1001 = new JPanel();
    panelArray_1001.setLayout (new FlowLayout());

    //panel kontrol
    JPanel controlPanel_1001 = new JPanel ();
    stepButton_1001 = new JButton ("Langkah
Selanjutnya");
    resetButton_1001 = new JButton ("Reset");

```

```

        stepButton_1001.setEnabled(false);
        controlPanel_1001.add(stepButton_1001);
        controlPanel_1001.add(resetButton_1001);

        //Area teks untuk log langkah-langkah
        stepArea_1001 = new JTextArea (8,60);
        stepArea_1001.setEditable(false);
        stepArea_1001.setFont(new Font ("Monospaced",
Font.PLAIN,14));
        JScrollPane scrollPane_1001 = new JScrollPane
(stepArea_1001);

        //Tambahkan panel ke frame
        add(inputPanel_1001, BorderLayout.NORTH);
        add(panelArray_1001, BorderLayout.CENTER);
        add(controlPanel_1001,BorderLayout.SOUTH);
        add(scrollPane_1001, BorderLayout.EAST);

        //event Set Array
        setButton_1001.addActionListener ( e ->
setArrayFromInput_1001());

        //event Langkah Selanjutnya
        stepButton_1001.addActionListener ( e->
performStep_1001());

        //event Reset
        resetButton_1001.addActionListener(e ->
reset_1001());
    }

    private void setArrayFromInput_1001() {
        String text_1001 = inputField_1001.getText().trim();
        if ( text_1001.isEmpty()) return;
        String [] parts_1001 = text_1001.split(",");
        array_1001 = new int [parts_1001.length];

        try {
            for ( int k_1001 = 0; k_1001 <
parts_1001.length; k_1001++) {
                array_1001 [k_1001] =
Integer.parseInt(parts_1001[k_1001].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka yang dipisahkan" + " dengan koma !" , "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_1001 = 1;
        stepCount_1001 =1;
        sorting_1001 = true;
        stepButton_1001.setEnabled(true);
        stepArea_1001.setText("");
        panelArray_1001.removeAll();
        labelArray_1001 = new JLabel [array_1001.length];
    }

```

```

        for (int k_1001 = 0; k_1001 < array_1001.length;
k_1001++) {
            labelArray_1001[k_1001] = new JLabel
(String.valueOf(array_1001[k_1001]));
            labelArray_1001[k_1001].setFont(new Font
("Arial", Font.BOLD,24));

            labelArray_1001[k_1001].setBorder(BorderFactory.createLineB
order(Color.BLACK));
            labelArray_1001[k_1001].setPreferredSize(new
Dimension (50,50));

            labelArray_1001[k_1001].setHorizontalAlignment(SwingConstan
ts.CENTER);
            panelArray_1001.add(labelArray_1001[k_1001]);
        }
        panelArray_1001.revalidate();
        panelArray_1001.repaint();
    }

```

2.5.2 Penjelasan

1. Rincian Objek GUI yang digunakan

- a. JFrame : mengatur properti dasar.
- b. JPanel : sebagai wadah untuk mengelompokkan komponen GUI. Dalam program ini variabel yang menggunakan JPanel adalah panelArray, inputPanel dan juga controlPanel.
- c. inputField_1001 : untuk memasukkan deret angka.
- d. setButton_1001 : memicu metode.
- e. setArrayFromInput_1001() : untuk memproses input.
- f. stepButton_1001 : memicu method performStep untuk menjalankan satu iterasi sorting.
- g. resetButton_1001 : memicu method reset_1001 untuk mengembalikan aplikasi ke keadaan awal.
- h. JTextArea: menampilkan log/catatan langkah algoritma secara detail.
- i. labelArray_1001 : memvisualisasikan setiap elemen integer menjadi kotak visual di layar.
- j. BorderLayout : mengatur tata letak utama frame ke posisi yang diinginkan.

- k. `FlowLayout` : mengatur tata letak di dalam panel- panel agar komponen tersusun rata tengah atau berurutan secara horizontal.
2. Rincian Variabel dan Method yang digunakan
- a. Variabel
 - `Array_1001` : menyimpan data integer asli yang akan diproses.
 - `labelArray_1001` : referensi ke komponen visual untuk update tampilan tanpa membuat ulang objek.
 - `i_1001, j_1001` : menyimpan posisi indeks algoritma insertion Sort agar proses bisa dihentikan dan dilanjutkan.
 - `sorting_1001` menandai apakah proses sorting sedang aktif atau sudah selesai.
 - `stepCount_1001` : penghitung nomor langkah untuk ditampilkan di log.
 - b. Constructor (`InsertionGUI_1001()`) : mengambil string dari `inputField_1001` dan memecahnya dengan `split (“,”)` dan mengonversinya menjadi array integer.
 - c. `performStep_1001` : menjalankan satu siklus logika
 - d. `updateLabels_1001` : sinkronisasi data dan tampilan.
 - e. `reset_1001` : membersihkan seluruh state aplikasi.
 - f. `arrayToString_1001` : konversi data.

2.5.3 Output

Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	5	9
6	4	1
8	7	3
	10	

Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	5	9
6	4	1
8	7	3
	10	

Langkah 1: Memasukkan 5
Hasil: 2, 5, 9, 6, 4, 1, 8, 7, 3, 10

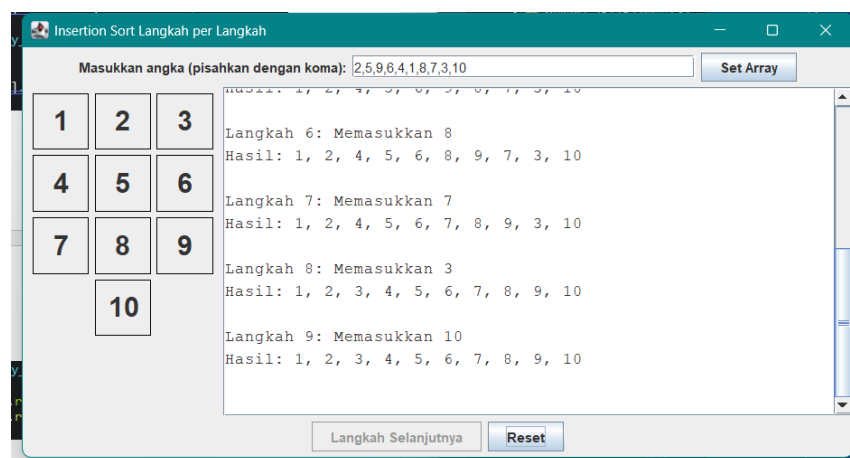
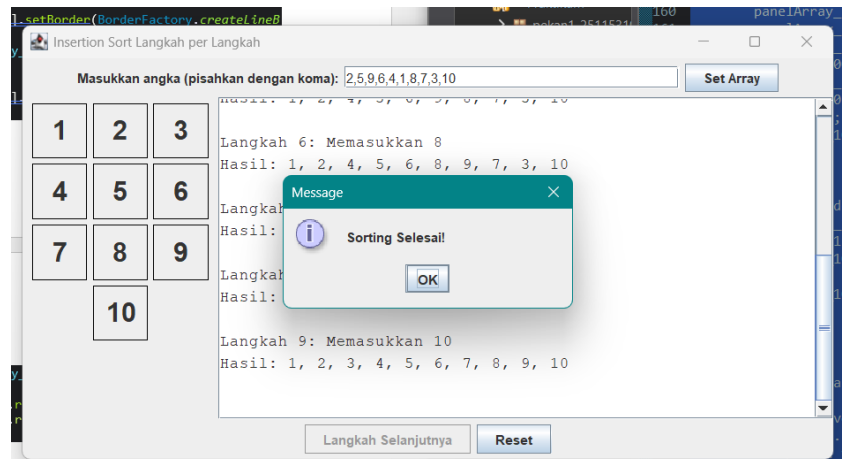
Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	5	9
6	4	1
8	7	3
	10	

Langkah 1: Memasukkan 5
Hasil: 2, 5, 9, 6, 4, 1, 8, 7, 3, 10

Langkah 2: Memasukkan 9
Hasil: 2, 5, 9, 6, 4, 1, 8, 7, 3, 10



BAB III

KESIMPULAN

Dari praktikum yang telah dilaksanakan, ada 3 poin yang dapat disimpulkan :

1. Insertion Sort

Algoritma ini bekerja dengan prinsip penyisipan elemen kunci ke dalam bagian array yang sudah terurut melalui mekanisme pergeseran, yang menjadikannya sangat efisien dengan kompleksitas $O(n)$ pada data yang hampir terurut serta bersifat stabil. Namun, karena sifatnya yang memindahkan elemen satu per satu, kinerjanya menjadi kurang optimal dengan kompleksitas $O(n^2)$ saat mengolah data dalam jumlah besar yang acak.

2. Selection Sort

Selection Sort memiliki ciri khas unik yaitu meminimalkan jumlah operasi pertukaran data dengan cara mencari nilai ekstrem untuk langsung ditukar ke posisi akhirnya, sehingga sangat berguna jika biaya swap data sangat mahal. Sayangnya, algoritma ini selalu memiliki kompleksitas waktu $O(n^2)$ tanpa memandang kondisi awal data dan bersifat unstable, sehingga tidak direkomendasikan untuk data yang membutuhkan preservasi urutan relatif.

Bubble Sort

3. Bubble Sort menggunakan logika paling sederhana yaitu membandingkan dan menukar elemen yang berdekatan agar nilai terbesar "mengapung" ke ujung array, yang membuatnya mudah dipahami namun sangat tidak efisien karena banyaknya operasi *swap* yang berulang. Meskipun dapat mencapai kompleksitas $O(n)$ pada data yang sudah terurut jika menggunakan optimasi, algoritma ini umumnya memiliki kinerja terendah secara praktis dibandingkan dua algoritma lainnya untuk kasus data acak.

DAFTAR PUSTAKA

LAPORAN TUGAS

A. Program Mahasiswa_2511531001

```
package pekan7_2511531001;
public class Mahasiswa_2511531001 {
    private String nama_1001;
    private String nim_1001;
    private String prodi_1001;

    //konstruktor
    public Mahasiswa_2511531001 (String n_1001, String
nim_1001, String p_1001) {
        this.nama_1001 = n_1001;
        this.nim_1001 = nim_1001;
        this.prodi_1001 = p_1001;
    }

    //getter
    public String getNama_1001() {return nama_1001;}
    public String getNim_1001() {return nim_1001;}
    public String getProdi_1001() {return prodi_1001;}

    //setter
    public void setNama_1001 (String nama_1001) {this.nama_1001
= nama_1001;}
    public void setNim_1001 (String nim_1001) {this.nim_1001 =
nim_1001;}
    public void setProdi_1001 (String prodi_1001)
{this.prodi_1001 = prodi_1001;}

    //tampilan data
    public String toString () {
        return nama_1001 + " " + nim_1001 + " " +
prodi_1001;
    }
}
```

B. Program SortingGUI_2511531001

// Package deklarasi untuk organisasi kode modul pekan 7

```
package pekan7_2511531001;
```

// Import library Java Swing untuk komponen GUI

```
import java.awt.BorderLayout; import java.awt.Color; import
java.awt.Dimension; // ... (dan import lainnya)
```

// Deklarasi kelas utama yang mewarisi JFrame sebagai window aplikasi

```
public class SortingGUI_2511531001 extends JFrame {
```

```

private static final long serialVersionUID = 1L; // ID versi untuk
kompatibilitas serialisasi Java Swing

// === Deklarasi Variabel Instance ===
private JTextField txtNama_1001, txtNim_1001, txtProdi_1001; // Field
input teks untuk data mahasiswa
private JButton btnTambah_1001, btnHapus_1001, btnSort_1001,
btnStep_1001, btnReset_1001; // Tombol kontrol aksi pengguna
private JComboBox<String> comboAlgoritma_1001; // Dropdown
untuk memilih jenis algoritma sorting
private JTable table_1001; // Komponen tabel untuk menampilkan
daftar data
private DefaultTableModel tableModel_1001; // Model data yang
mengelola isi baris dan kolom tabel
private JTextArea logArea_1001; // Area teks untuk menampilkan log
proses sorting langkah demi langkah
private ArrayList<Mahasiswa_2511531001> dataList_1001; // List
utama penyimpan data mahasiswa asli
private ArrayList<Mahasiswa_2511531001> workingList_1001; // List
salinan untuk manipulasi data saat sorting
private String selectedAlgorithm_1001 = ""; // Variabel penyimpan
nama algoritma yang sedang aktif
private boolean isSorting_1001 = false; // Flag status untuk menandai
apakah proses sorting sedang berjalan

// Variabel state khusus untuk melacak posisi iterasi Insertion Sort
private int insertionI_1001, insertionJ_1001; private
Mahasiswa_2511531001 insertionKey_1001; private int
insertionPhase_1001;

// Variabel state khusus untuk melacak posisi iterasi Selection Sort

```

```

private int selectionI_1001, selectionJ_1001, selectionMinIdx_1001;
private int selectionPhase_1001;

// Variabel state khusus untuk melacak posisi iterasi Bubble Sort
private int bubbleI_1001, bubbleJ_1001; private int bubblePhase_1001;
private int stepCounter_1001 = 1; // Penghitung nomor langkah untuk
ditampilkan di log area

// === Constructor: Inisialisasi GUI ===
public SortingGUI_2511531001() {
    setTitle("Sorting Mahasiswa - Insertion | Selection | Bubble Sort"); //
Mengatur judul jendela aplikasi
    setSize(900,600); // Mengatur dimensi lebar dan tinggi window
dalam pixel
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Menutup
aplikasi sepenuhnya saat window ditutup
    setLocationRelativeTo(null); // Memposisikan window muncul tepat
di tengah layar monitor
    setLayout(new BorderLayout(10,10)); // Mengatur layout manager
utama dengan jarak antar komponen 10px
    dataList_1001 = new ArrayList<>(); workingList_1001 = new
ArrayList<>(); // Inisialisasi list data kosong

// --- Panel Input Data ---
JPanel inputPanel_1001 = new JPanel (new FlowLayout
(FlowLayout.LEFT,10,10)); // Panel container untuk form input
    inputPanel_1001.setBorder(BorderFactory.createTitledBorder("Input
Data Mahasiswa")); // Memberi judul berbingkai pada panel
    txtNama_1001 = new JTextField(15); txtNim_1001 = new
JTextField(15); txtProdi_1001 = new JTextField(15); // Buat field input
    btnTambah_1001 = new JButton("Tambah"); btnHapus_1001 = new
JButton ("Hapus Terpilih"); // Inisialisasi tombol CRUD

```

```

        inputPanel_1001.add(new JLabel("Nama:"));
inputPanel_1001.add(txtNama_1001); // Tambahkan label dan field ke
panel

// ... (penambahan komponen input lainnya ke panel) ...

// --- Panel Kontrol Sorting ---
JPanel controlPanel_1001 = new JPanel( new FlowLayout
(FlowLayout.CENTER, 15,10)); // Panel untuk tombol sorting

controlPanel_1001.setBorder(BorderFactory.createTitledBorder("Kontrol
Sorting")); // Judul panel kontrol

        comboAlgoritma_1001 = new JComboBox<>(new String [] {"Pilih
Algoritma", "Insertion Sort", "Selection Sort", "Bubble Sort"}); // Opsi
algoritma

        btnSort_1001 = new JButton ("Mulai Sorting"); btnStep_1001 = new
JButton ("Langkah Selanjutnya"); btnReset_1001 = new JButton
("Reset"); // Tombol aksi

        btnStep_1001.setEnabled(false); btnSort_1001.setEnabled(false); //
Nonaktifkan tombol saat awal aplikasi

        controlPanel_1001.add(new JLabel ("Algoritma"));
controlPanel_1001.add(comboAlgoritma_1001); // Susun komponen
kontrol

// ... (penambahan tombol ke panel kontrol) ...

// --- Tabel Data ---

String [] columns_1001 = {"NO", "Nama", "NIM", "Program
Studi"}; // Definisi header kolom tabel

        tableModel_1001 = new DefaultTableModel (columns_1001, 0)
{ public boolean isCellEditable (int row, int column) { return false; } }; //
Model tabel readonly

        table_1001 = new JTable (tableModel_1001); // Membuat instance
JTable dengan model tersebut

```

```

table_1001.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_S
ELECTION); // Izinkan hanya satu baris terpilih saat klik
    table_1001.setRowHeight(25); table_1001.setFont(new Font ("Arial",
Font.PLAIN, 12)); // Styling visual tabel
    JScrollPane tableScroll_1001 = new JScrollPane (table_1001); //
Membungkus tabel agar bisa discroll
    tableScroll_1001.setPreferredSize(new Dimension (850,150)); //
Mengatur tinggi area tabel

    // --- Area Log ---
    logArea_1001 = new JTextArea (12,70);
logArea_1001.setEditable(false); // Area teks readonly untuk output log
    logArea_1001.setFont(new Font ("Monospaced", Font.PLAIN, 11));
logArea_1001.setBackground(new Color (245, 245, 245)); // Styling font
dan background log
    JScrollPane logScroll_1001 = new JScrollPane (logArea_1001);
logScroll_1001.setBorder(BorderFactory.createTitledBorder("Log Proses
Sorting")); // Scrollable log area

    // --- Penyusunan Layout Utama ---
    JPanel topPanel_1001 = new JPanel (new BorderLayout (10,10));
topPanel_1001.add(inputPanel_1001, BorderLayout.NORTH); // Panel
atas berisi input
    JPanel centerPanel_1001 = new JPanel (new BorderLayout (10,10));
centerPanel_1001.add(tableScroll_1001, BorderLayout.NORTH);
centerPanel_1001.add(logScroll_1001, BorderLayout.CENTER); // Panel
tengah berisi tabel dan log
    add (topPanel_1001, BorderLayout.NORTH); add
(centerPanel_1001, BorderLayout.CENTER); add (controlPanel_1001,
BorderLayout.SOUTH); // Tambahkan panel ke frame utama

```



```

// --- Registrasi Event Listener (Lambda) ---
btnTambah_1001.addActionListener(e -> tambahData_1001()); //
Event klik tombol Tambah
btnHapus_1001.addActionListener(e -> hapusdata_1001()); // Event
klik tombol Hapus
comboAlgoritma_1001.addActionListener(e ->
updateButtonState_1001()); // Event perubahan pilihan algoritma
btnSort_1001.addActionListener(e -> mulaiSorting_1001()); // Event
klik tombol Mulai Sorting
btnStep_1001.addActionListener(e -> lakukanLangkah_1001()); //
Event klik tombol Langkah Selanjutnya
btnReset_1001.addActionListener(e -> reset_1001()); // Event klik
tombol Reset
}

// === Method Logika Bisnis ===
private void tambahData_1001() { // Method untuk menangani
penambahan data baru dari input form
    String nama_1001 = txtNama_1001.getText().trim(); String
nim_1001 = txtNim_1001.getText().trim(); String prodi_1001 =
txtProdi_1001.getText().trim(); // Ambil dan bersihkan teks input
    if (nama_1001.isEmpty() || nim_1001.isEmpty() ||
prodi_1001.isEmpty()) { JOptionPane.showMessageDialog(this, "Semua
field harus diisi", "Input Error", JOptionPane.WARNING_MESSAGE);
return; } // Validasi input tidak boleh kosong
    Mahasiswa_2511531001 mhs_1001 = new Mahasiswa_2511531001
(nama_1001, nim_1001, prodi_1001); dataList_1001.add(mhs_1001);
refreshTable_1001(); // Buat objek, simpan ke list, update tabel
    txtNama_1001.setText(""); txtNim_1001.setText("");
txtProdi_1001.setText(""); txtNama_1001.requestFocus(); // Reset form
dan fokus ke field pertama
    updateButtonState_1001(); // Perbarui status enabled/disabled tombol

```

```

    }

    private void hapusdata_1001() { // Method untuk menghapus data yang
    terpilih di tabel

        int selectedRow_1001 = table_1001.getSelectedRow(); if
    (selectedRow_1001 == -1) { JOptionPane.showMessageDialog(this, "Pilih
    data yang ingin dihapus!", "warning",
    JOptionPane.WARNING_MESSAGE); return; } // Cek apakah ada baris
    terpilih

        dataList_1001.remove(selectedRow_1001); refreshTable_1001();
    updateButtonState_1001(); // Hapus dari list, refresh UI

    }

    private void refreshTable_1001() { // Method helper untuk memperbarui
    tampilan tabel sesuai isi dataList_1001

        tableModel_1001.setRowCount(0); // Kosongkan semua baris tabel
    yang ada

        for (int i_1001 = 0; i_1001 < dataList_1001.size(); i_1001++)
    { Mahasiswa_2511531001 m_1001 = dataList_1001.get(i_1001);
    tableModel_1001.addRow(new Object [] {(i_1001 + 1),
    m_1001.getNama_1001(),
    m_1001.getNim_1001(),m_1001.getProdi_1001}); } // Loop isi data ke
    baris tabel

    }

    private void updateButtonState_1001 () { // Method untuk mengatur
    status aktif/tidak tombol berdasarkan kondisi data dan pilihan

        boolean hasData_1001 = !dataList_1001.isEmpty(); String
    selected_1001 = (String) comboAlgoritma_1001.getSelectedItem();
    btnSort_1001.setEnabled(hasData_1001 && !selected_1001.equals("Pilih
    Algoritma")); // Aktifkan btnSort hanya jika ada data dan algoritma dipilih

    }

```

```

private void mulaiSorting_1001 () { // Method inialisasi sebelum
proses sorting dimulai
    if (dataList_1001.isEmpty()) return; // Guard clause jika data kosong
    selectedAlgorithm_1001 = (String)
comboAlgoritma_1001.getSelectedItem(); workingList_1001 = new
ArrayList<> ( dataList_1001); // Copy data asli ke working list
    isSorting_1001 = true; stepCounter_1001 = 1;
    btnStep_1001.setEnabled(true); btnSort_1001.setEnabled(false);
    comboAlgoritma_1001.setEnabled(false); logArea_1001.setText(""); //
Reset state UI dan log
    logArea_1001.append("====" +
selectedAlgorithm_1001.toUpperCase() + "====\n");
    logArea_1001.append ("data awal : "+ listToString_1001
(workingList_1001)+ "\n"); // Cetak header log
    // Inialisasi variabel state spesifik sesuai algoritma terpilih
    if (selectedAlgorithm_1001.equals("Insertion Sort"))
    { insertionI_1001 = 1; insertionPhase_1001 = 0; } else if
(selectedAlgorithm_1001.equals("Selection Sort")) { selectionI_1001 = 0;
selectionJ_1001 = 1; selectionPhase_1001 = 0; } else if
(selectedAlgorithm_1001.equals("Bubble Sort")){ bubbleI_1001 = 0;
bubbleJ_1001 = 0; bubblePhase_1001 = 0; }
    refreshTable_1001(); // Refresh tampilan awal
}

private void lakukanLangkah_1001() { // Method dispatcher untuk
menjalankan satu langkah sorting
    if (!isSorting_1001) return; // Cek flag status sorting
    switch (selectedAlgorithm_1001) { case "Insertion Sort":
langkahInsertion_1001(); break; case "Selection Sort":
langkahSelection_1001(); break; case "Bubble Sort":
langkahBubble_1001(); break; } // Panggil method sesuai pilihan

```

```

        refreshTable_1001(); // Update tabel setelah satu langkah dieksekusi
    }

    private void langkahInsertion_1001() { // Implementasi logika satu
langkah Insertion Sort
        if (insertionI_1001 >= workingList_1001.size())
        { selesaiSorting_1001(); return; } // Cek kondisi selesai
        if (insertionPhase_1001 == 0) { insertionKey_1001 =
workingList_1001.get(insertionI_1001); insertionJ_1001 =
insertionI_1001 - 1; logArea_1001.append("Langkah " +
stepCounter_1001 + ": Memproses \"" +
insertionKey_1001.getNama_1001() + "\"\n"); insertionPhase_1001 = 1; }
// Fase 0: Ambil key
        else if (insertionPhase_1001 == 1) { if (insertionJ_1001 >= 0 &&
workingList_1001.get(insertionJ_1001).getNama_1001().compareToIgnor
eCase(insertionKey_1001.getNama_1001()) > 0) { workingList_1001.set
(insertionJ_1001 + 1, workingList_1001.get(insertionJ_1001));
logArea_1001.append(" -> Shift \"" +
workingList_1001.get(insertionJ_1001 + 1).getNama_1001() + "\" ke
kanan\n"); insertionJ_1001--; } else { insertionPhase_1001 = 2; return; } }
// Fase 1: Geser elemen
        else if (insertionPhase_1001 == 2)
        { workingList_1001.set(insertionJ_1001 + 1, insertionKey_1001);
logArea_1001.append(" -> Insert \""
+insertionKey_1001.getNama_1001() + "\" di posisi " + (insertionJ_1001
+ 2) + "\n"); logArea_1001.append(("Hasil Sementara: " +
listToString_1001(workingList_1001) + "\n\n")); insertionI_1001++;
insertionPhase_1001 = 0; stepCounter_1001++; if (insertionI_1001 >=
workingList_1001.size()) { selesaiSorting_1001(); } } // Fase 2: Sisipkan
key
    }
}

```

```

private void langkahSelection_1001() { // Implementasi logika satu
langkah Selection Sort

    if (selectionI_1001 >= workingList_1001.size()-1)
    { selesaiSorting_1001(); return; } // Cek kondisi selesai

    if (selectionPhase_1001 == 0) { if (selectionJ_1001 ==
selectionI_1001 + 1) { selectionMinIdx_1001 = selectionI_1001;
logArea_1001.append("Langkah" + stepCounter_1001 + ": Pass" +
(selectionI_1001 + 1) + " - Mencari minimum dari index " +
selectionI_1001 + "\n"); } if (selectionJ_1001 < workingList_1001.size())
{ if
(workingList_1001.get(selectionJ_1001).getNama_1001().compareToIgno
reCase(workingList_1001.get(selectionMinIdx_1001).getNama_1001())<
0) { selectionMinIdx_1001 = selectionJ_1001; logArea_1001.append(" ->
Ditemukan angka yang lebih kecil\n"+
workingList_1001.get(selectionMinIdx_1001).getNama_1001()+ "\n\n"); }
selectionJ_1001++; } else { selectionPhase_1001 = 1 ; selectionJ_1001 =
selectionI_1001; return; } } // Fase 0: Cari min

    else if (selectionPhase_1001 == 1) { if (selectionMinIdx_1001 !=
selectionI_1001) { Mahasiswa_2511531001 temp_1001 =
workingList_1001.get(selectionI_1001);
workingList_1001.set(selectionI_1001,
workingList_1001.get(selectionMinIdx_1001));
workingList_1001.set(selectionMinIdx_1001, temp_1001);
logArea_1001.append(" -> Swap: \n" +
workingList_1001.get(selectionI_1001).getNama_1001()+"\n <-> \n" +
workingList_1001.get(selectionMinIdx_1001).getNama_1001()+ "\n\n"); }
logArea_1001.append("Hasil Pass " + (selectionI_1001 + 1) + ": "+
listToString_1001(workingList_1001) + "\n\n"); selectionI_1001++;
selectionJ_1001 = selectionI_1001 + 1; selectionPhase_1001 = 0;
stepCounter_1001++; if (selectionI_1001 >= workingList_1001.size()-1)
{ selesaiSorting_1001(); } } // Fase 1: Swap

    }

```

```

private void langkahBubble_1001() { // Implementasi logika satu
langkah Bubble Sort
    if (bubbleI_1001 >= workingList_1001.size()-1)
    { selesaiSorting_1001(); return; } // Cek kondisi selesai
    if (bubblePhase_1001 == 0) { if (bubbleJ_1001 == 0)
    { logArea_1001.append("Langkah" + stepCounter_1001 + ": Pass" +
    (bubbleI_1001 + 1) + "\n"); } String namaKiri_1001 =
    workingList_1001.get(bubbleJ_1001).getNama_1001(); String
    namaKanan_1001 = workingList_1001.get(bubbleJ_1001+
    1).getNama_1001(); logArea_1001.append(" -> Bandingkan \"" +
    namaKiri_1001 + "\" vs \"" + namaKanan_1001 + "\"\n"); if
    (namaKiri_1001.compareToIgnoreCase(namaKanan_1001)>0)
    { bubblePhase_1001 = 1; } else { bubbleJ_1001++; if (bubbleJ_1001 >=
    workingList_1001.size()-1-bubbleI_1001) { logArea_1001.append(("Hasil
    Pass " + (bubbleI_1001 + 1) + ": " +
    listToString_1001(workingList_1001)+ "\n\n")); bubbleI_1001++;
    bubbleJ_1001 = 0;; stepCounter_1001++; if (bubbleI_1001 >=
    workingList_1001.size()-1) { selesaiSorting_1001(); } } } // Fase 0:
    Bandingkan
    else if (bubblePhase_1001 == 1) { Mahasiswa_2511531001
    temp_1001 = workingList_1001.get(bubbleJ_1001);
    workingList_1001.set(bubbleJ_1001,
    workingList_1001.get(bubbleJ_1001+ 1));
    workingList_1001.set(bubbleJ_1001 + 1, temp_1001);
    logArea_1001.append("-> Swap dilakukan! \n"); bubblePhase_1001 =0;
    bubbleJ_1001 ++; if (bubbleJ_1001 >= workingList_1001.size()-1-
    bubbleI_1001) { logArea_1001.append("Hasil Pass " + ( bubbleI_1001 +
    1) + ": " + listToString_1001(workingList_1001) + "\n\n"));
    bubbleI_1001++; bubbleJ_1001 = 0; stepCounter_1001 ++; if
    (bubbleI_1001 >= workingList_1001.size() - 1)
    { selesaiSorting_1001(); } } } // Fase 1: Swap

```

```

    }

    private void selesaiSorting_1001() { // Method handler saat sorting
selesai total

        logArea_1001.append("\n SORTING SELESAI!\n");
logArea_1001.append("Hasil Akhir (A-Z): " +
listToString_1001(workingList_1001) + "\n"); // Cetak hasil final ke log
        isSorting_1001 = false; btnStep_1001.setEnabled(false);
btnSort_1001.setEnabled(true); comboAlgoritma_1001.setEnabled(true);
// Reset state UI
        JOptionPane.showMessageDialog(this, "Sorting berhasil selesai
diselesaikan", "Selesai", JOptionPane.INFORMATION_MESSAGE); //
Tampilkan dialog sukses
    }

    private void reset_1001 () { // Method untuk mereset seluruh aplikasi ke
kondisi awal

        dataList_1001.clear(); workingList_1001.clear();
refreshTable_1001(); logArea_1001.setText(""); // Bersihkan data dan UI
        comboAlgoritma_1001.setSelectedIndex(0);
btnStep_1001.setEnabled(false); btnSort_1001.setEnabled(false);
comboAlgoritma_1001.setEnabled(true); isSorting_1001 = false; // Reset
kontrol

        txtNama_1001.setText(""); txtNim_1001.setText("");
txtProdi_1001.setText(""); // Kosongkan form input
    }

    private String listToString_1001 (ArrayList<Mahasiswa_2511531001>
list_1001) { // Helper method konversi list ke string format log
        StringBuilder sb_1001 = new StringBuilder(""); for (int i_1001 = 0;
i_1001 < list_1001.size(); i_1001++)
        { sb_1001.append("\n").append(list_1001.get(i_1001).getNama_1001()).a

```

```

ppend("\n"); if (i_1001 < list_1001.size()-1) sb_1001.append(", "); }
sb_1001.append("]"); return sb_1001.toString(); // Loop dan format string
}

```

```

// === Main Method ===

```

```

public static void main (String [] args) { EventQueue.invokeLater() ->
{ try { SortingGUI_2511531001 frame = new SortingGUI_2511531001 ();
frame.setVisible(true); } catch (Exception e) { e.printStackTrace(); } }); }
// Entry point: Jalankan GUI di Event Dispatch Thread
}

```

C. Ouput

1. Tampilan Awal

The screenshot shows a Java Swing window titled "Sorting Mahasiswa - Insertion | Selection | Bubble Sort". The window is divided into several sections:

- Input Data Mahasiswa:** This section contains three text input fields labeled "Nama:", "NIM:", and "Prodi:". To the right of these fields are two buttons: "Tambah" and "Hapus Terpilih".
- Table:** Below the input fields is a table with four columns: "NO", "Nama", "NIM", and "Program Studi". The table is currently empty.
- Log Proses Sorting:** This section is a large, empty text area intended for displaying the steps of the sorting process.
- Kontrol Sorting:** This section is located at the bottom of the window. It includes a label "Algoritma" followed by a dropdown menu currently set to "Pilih Algoritma". To the right of the dropdown are three buttons: "Mulai Sorting", "Langkah Selanjutnya", and "Reset".

2. Insertion Sort

Sorting Mahasiswa - Insertion | Selection | Bubble Sort

Input Data Mahasiswa

Nama: NIM: Prodi:

NO	Nama	NIM	Program Studi
1	zaza	1001	informatika
2	budi	3004	akuntansi
3	caca	2018	Hukum

Log Proses Sorting

```

-> Shift "zaza" ke kanan
-> Insert "budi" di posisi 1
Hasil Sementara: ["budi", "zaza", "caca"]

Langkah 2: Memproses "caca"
-> Shift "zaza" ke kanan
-> Insert "caca" di posisi 2
Hasil Sementara: ["budi", "caca", "zaza"]

SORTING SELESAI!
Hasil Akhir (A-Z): ["budi", "caca", "zaza"]

```

Kontrol Sorting

Algoritma

3. Selection Sort

Sorting Mahasiswa - Insertion | Selection | Bubble Sort

Input Data Mahasiswa

Nama: NIM: Prodi:

NO	Nama	NIM	Program Studi
1	zaza	1001	informatika
2	budi	3004	hukum
3	jery	2017	pendidikan dokter

Log Proses Sorting

```

-> Swap dilakukan!
-> Bandingkan "zaza" vs "jery"
-> Swap dilakukan!
Hasil Pass 1: ["budi", "jery", "zaza"]

Langkah2: Pass2
-> Bandingkan "budi" vs "jery"
Hasil Pass 2: ["budi", "jery", "zaza"]

SORTING SELESAI!
Hasil Akhir (A-Z): ["budi", "jery", "zaza"]

```

Kontrol Sorting

Algoritma

4. Bubble Sort

Sorting Mahasiswa - Insertion | Selection | Bubble Sort

Input Data Mahasiswa

Nama: NIM: Prodi:

NO	Nama	NIM	Program Studi
1	zaza	1001	informatika
2	budi	3004	hukum
3	jery	2017	pendidikan dokter

Log Proses Sorting

```
--> Swap dilakukan!  
--> Bandingkan "zaza" vs "jery"  
--> Swap dilakukan!  
Hasil Pass 1: ["budi", "jery", "zaza"]  
  
Langkah2: Pass2  
--> Bandingkan "budi" vs "jery"  
Hasil Pass 2: ["budi", "jery", "zaza"]  
  
SORTING SELESAI!  
Hasil Akhir (A-Z): ["budi", "jery", "zaza"]
```

Kontrol Sorting

Algoritma